
TIBER CAD

Examples Book *Release 2.0*

TiberLAB srl

April 1, 2011

CONTENTS

I	Examples	1
1	Elasticity	3
1.1	Piezoelectric nanogenerator	3
2	Drift Diffusion	7
2.1	Tutorial Bulk Si	7
2.2	Tutorial Si-Pn Diode	13
2.3	n-MosFET 2D	22
2.4	Bjt1 & Bjt2	30
3	Thermal	39
3.1	Boundary conditions	39
3.2	Heat sources	40
4	Input for TiberCAD	43
4.1	Description of Input file structure	43
4.2	Device section	44
4.3	Modules	46
4.4	Simulation section	50
4.5	Output description	51
4.6	Example of Input file	51
5	Simulation Dye Solar Cells	57
5.1	Device structure	58
II	Glossary	65
	Bibliography	69
	Index	71

Part I

Examples

ELASTICITY

1.1 Piezoelectric nanogenerator

In this example we will show how to perform electro-mechanical simulation of a piezoelectric nanogenerators [Wang] . The simulation domain consists of a Zinc oxide nanowire (length = 200 nm, diameter = 50 nm) growth on a ZnO substrate and surrounded by air. The performance of such a device can be obtained by computed the piezopotential produced when the nanowire is subjected to a shear force.

With the block device we specify the materials and properties associated with the physical regions. In particular, we have the *Nanowire*, *Substrate* and *Air* .

Device

```
{
  dimension = 3
  meshfile = mesh.msh
  mesh_units = 1e-9
  material = AlGaIn
  x = 0.2
  x-growth-direction = (-1,0,1,0)
  y-growth-direction = (-1,2,-1,0)
  z-growth-direction = (0,0,0,1)
  Region Nanowire
  {
    doping_type = donors
    doping_level = 0.035
  }
  Region Substrate
  {
    doping_type = donors
    doping_level = 0.035
  }
  Region Air
  {
```

```
        material = Air
    }
}
```

In this example we use a isotropic stiffness constant and, therefore, a Young modulus and Poisson's ration must be defined. The base of the substrate is clamped whereas a shear force is applied on the top of the nanowire.

```
Module elasticity
{
    Physics
    {
        Stiffness isotropic
        {
            Young = 129 Poisson = 0.349
        }
        Contact Base
        {
            type = clamp
        }
        Contact Top
        {
            type = surface_force force = (0.0, 0.0625,0.0)
        }
    }
}
```

The driftdiffusion module computes the piezopotential due to the applied force. The piezopolarization uses the strain computed by the elasticity module. The base of the substrate is grounded.

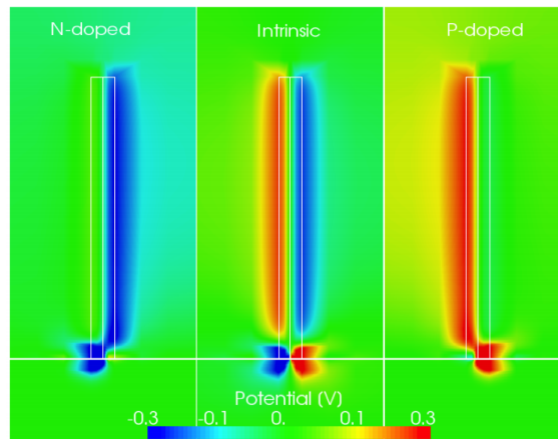
```
Module driftdiffusion
{
    Physics
    {
        {
            strain_simulation = elasticity
        }
        Contact Base
        {
            Voltage = 0.0
        }
        Polarization piezo
        {
        }
    }
}
```

In the simulations block we specify the simulations to be solved. The default simulation name is the name of the module itself.

Simulations

```
{  
  solve (elasticity,driftdiffusion)  
}
```

Consistently with results published in [Gao] the free carriers tend to screen the output potential. The potential profile is shown below.



DRIFT DIFFUSION

2.1 Tutorial Bulk Si

In this example we will see a very simple TiberCAD simulation:

1D calculation of Poisson and drift-diffusion for a bulk Silicon sample.

The following files should be in your working directory:

`bulk.tib` : **TiberCAD** input file

`bulk.msh` : mesh file

The mesh file can be obtained from the following GMSH geo file : `bulk.geo`

This is the summary of the Sections of this Tutorial :

- *Step 1 - Modeling the device*
- *Step 2 - Meshing the device*
- *Step 3 - TiberCAD Input file*
- *Step 4 - Run TiberCAD*
- *Output*

2.1.1 Step 1 - Modeling the device

As a first step, we have to model the device. To do so, you can use DEVISE module of ISE-TCAD 9.5 software package or GMSH program.

Here we'll see in details the procedure for GMSH.

There are two possible ways to use GMSH:

Interactive, using the graphical interface Using a script file In the following we'll see how to write a basic GMSH script (bulk.geo); for any details please refer to GMSH manual GMSH (<http://geuz.org/gmsh/>).



: In a **1D** simulation it is assumed that the geometrical model is restricted to the **x axis** . In a **2D** simulation it is assumed that the geometrical model is restricted to the **xy-plane (z=0)** Any other geometrical orientation could give unpredictable results

In a GMSH script, several variables can be defined and given a value in this way:

```
L = 1  
d = 0.01
```

these are valid GMSH variables: L is just the length of the Si sample; d is the value of a *characteristic mesh length* (see below).

Definition of geometrical entities **Points**

```
Point(1) = {0, 0, 0, d}  
Point(2) = {L, 0, 0, d}
```

The first three expressions inside the braces on the right hand side give the three X, Y and Z **coordinates** of the point; the last expression (**d**) sets the **characteristic mesh length** at that point, that is the “size” of a mesh element, defined as the length of the segment for a line segment, the radius of the circumscribed circle for a triangle and the radius of the circumscribed sphere for a tetrahedron.

Thus, the smaller is the value of d, the greater is the mesh density close to that point.

The size of the mesh elements will then be computed in GMSH by linearly interpolating these characteristic lengths in the whole mesh.

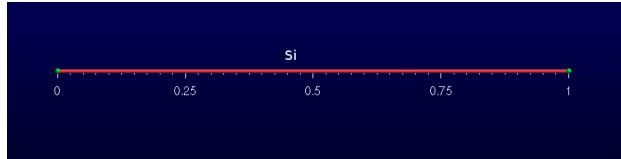
Definition of a geometrical entity **Line**

```
Line(1) = {1, 2}
```

The two expressions inside the braces on the right hand side give the identification numbers of the start and end points of the line.

Definition of the physical entity **Physical Line bulk**

```
Physical Line("bulk") = {1}
```



The expression(s) inside the braces on the right hand side give the identification numbers of all the **geometrical lines** that need to be gro ed inside the *physical line* .

In this way, in general, physical regions are created which associate together geometrical regions, and then the related mesh elements, which share some common physical properties. It's only these physical regions which can be referred to outside GMSH. In TiberCAD, this is done by associating one or more physical regions to a **TiberCAD region** through the keyword *mesh_regions* (see in the following).

Definition of two physical entities Physical Point:

```
Physical Point("anode2")    = {1}
Physical Point("cathode")   = {2}
```

|warn| : In general, in a nD simulation, ***(n-1)D*** physical regions (points in 1D, are used by TiberCAD to impose the required boundary conditions.

Each (n-1)D physical region defined in this way in GMSH will be associated in TiberCAD through the keyword **BC_reg_num**. Thus, in this case, Physical points Anode and Cathode are associated to two **Contact** regions (see in the following).

2.1.2 Step 2 - Meshing the device

The *.geo* script file with the geometrical description can be run in GMSH, to display the modelled device and to mesh it through the GMSH graphical interface. Alternatively, a *non-interactive* mode is also available in GMSH, without graphical user interface.

For example, to mesh this 1D tutorial in non-interactive mode, just type:



```
gmsht bulk.geo -1 -o bulk.msh
```

where *bulk.geo* is the geometrical description of the device with GMSH syntax:

-1 means *1D mesh generation*

some command line options are:

-1 , -2, -3 to perform *1D, 2D or 3D* mesh generation,

-o **mesh_file.msh** to specify the name of the mesh file to be generated

In this way, a *.msh* has been generated and is ready to be read in TiberCAD.

2.1.3 Step 3 - TiberCAD Input file

Now we have to write down the **TiberCAD input file** (`bulk.tib`). For a detailed reference see the user guide (Input file) and Getting started 1

2.1.4 Description of Device Regions

First, we have to list all the **TiberCAD Regions** present in our Device: a TiberCAD Region is usually a section of the device featuring the same material and possibly the same doping.

```
Device
{
    meshfile = bulk.msh

    Region bulk
    {
        material = Si

        Doping
        {
            Nd = 1e16
            type = donor
        }
    }
}
```

The TiberCAD Region bulk is made of Silicon and n-doped with a concentration $1 \times 10^{16} \text{ cm}^{-3}$.

Through the keyword **Region** , one GMSH physical region (Physical Lines in 1D, Physical Surfaces in 2D, Physical Volumes in 3D) previously defined in the GMSH mesh (*Step 1 - Modeling the device*), can be associated to the present TiberCAD Region, in this way:

```
Region  GMSH_physical_region_name
```

In this case, the Physical Line bulk is associated to the TiberCAD Region bulk.

Alternatively, through the optional keyword **mesh_regions** , one or more GMSH physical regions can be associated to a single TiberCAD Region .

2.1.5 Definition of Simulation

Now we define the **Simulation** driftdiffusion_1: it belongs to the class **driftdiffusion**

```
Module driftdiffusion
{
    name = driftdiffusion  # this is the default name
```

```
regions = all           # 'all' is the default

# what we want to plot
plot = (Ec, Ev, eQFermi, hQFermi, ContactCurrent)
....
```

The TiberCAD simulation *driftdiffusion_1* , belonging to the model *driftdiffusion*, will be applied to the whole device structure (`regions = all`)

2.1.6 Definition of Boundary Conditions

The anode and cathode contacts of our 1D Si sample are defined as **Boundary conditions regions** (`Contact anode`, `Contact cathode`) in the following way:

```
Module driftdiffusion {
    name = driftdiffusion
    regions = all
    plot = (Ec, Ev, eQFermi, hQFermi, ContactCurrent)
    Contact anode { voltage = $Vb } Contact cathode { }
}
```

Through the keyword `Contact` , one (n-1) -dimension GMSH physical region (Physical Point in 1D, Physical Line in 2D, Physical Surface in 3D) previously defined in the GMSH mesh ([Step 1 - Modeling the device](#)), can be associated to the present TiberCAD Contact, in this way:

`Contact GMSH_physical_region_name`

In this case, the *Physical Point* **anode** is associated to the TiberCAD Contact anode and the Physical Point cathode is associated to the TiberCAD Contact cathode.

Both contacts are defined as *ohmic* , cathod is assigned a fixed `voltage = 0.0` , while anode voltage is given by the value of the variable *Vb*

```
voltage = @Vb
```

2.1.7 Definition of Simulation parameters

Vb is specified in the sweep block, in the Solver section

```
Module sweep {
    solve = driftdiffusion variable = $Vb start = 0.0 stop = 1 steps = 10 plot_data = true
```

```
}
```

In this way, the simulation `driftdiffusion_1` is performed for 10 (steps = 10) values of the anode voltage (variable = `Vb`), between 0 and 1.

For each step we want to plot the solution variables specified in the `driftdiffusion` module (`plot_data = true`).

2.1.8 Definition of Execution parameters

In the **Simulation** section , we decide *which* simulations to perform and in which *order*; set `solve = sweep` , to execute the sweep which run `driftdiffusion_1` simulation for the specified loop.

```
Simulation
{
    verbose = 2

    solve = sweep

    resultpath = output
    output_format = grace
}
```

Output files with conduction and valence band profiles (`plot = Ec,Ev..`) and all the calculated values of the current at the contacts (*ContactCurrents*) (the IV characteristic) are generated.

To increase the amount of information written to the screen we can vary the verbose level (`verbose = 2`).

2.1.9 Step 4 - Run TiberCAD

Now we can run TiberCAD

by double clicking on `bulk.tib` file (in Windows)

or by command line in linux: `tibercad bulk.tib`

2.1.10 Output

The generated Output files are:

- `driftdiffusion_materials.dat` : material (mesh) regions, in this case just region 1

- driftdiffusion_nodal.dat : nodal quantities (here conduction and valence band)
- sweep_driftdiffusion_Vb.dat : integrated current at the two contacts for each sweep step

Attachment Size

bulk.tib 1.16 KB

bulk.geo 181 bytes

bulk.msh 4.49 KB

2.2 Tutorial Si-Pn Diode

In this first tutorial we will show a simple 1D Si pn diode example. We will calculate the IV characteristic of the pn diode, by solving the Poisson equation and calculating the current with a drift-diffusion scheme.

The semi-classical transport simulation of electrons and holes is based on the driftdiffusion approximation .

Beside the electric potential, the electro-chemical potentials are used as variables such that the system of PDEs to be solved reads as follows:

$$\begin{aligned} -\nabla(\varepsilon\nabla\varphi - \mathbf{P}) &= -e(n - p - N_d^+ + N_a^-) \\ -\nabla(\mu_n n \nabla\phi_n) &= R \\ -\nabla(\mu_p p \nabla\phi_p) &= -R \end{aligned} \tag{2.1}$$

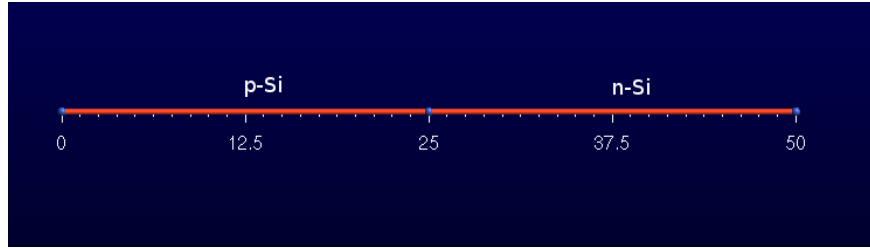
where $\mathbf{P}$ is the electric polarization due to e.g. piezoelectric effects and $\mathbf{R}$ is the net recombination rate, i.e. recombination rate minus generation rate.

In order to execute correctly the example you should have the following files in the same working directory:

diode_1D.tib : input file for TiberCAD diode_1D.msh : mesh file produced by GMSH (<http://geuz.org/gmsh/>).

GMSH is an automatic three-dimensional finite element mesh generator. Please refer to GMSH manual for any details.

The mesh has been obtained from the GMSH geo script file diode_1D.geo .



2.2.1 Modeling of the device with GMSH

In this script, first the geometrical entities Points and Lines are defined (see *Tutorial Bulk Si*). Then, two physical regions are defined, each associated to one of the two geometrical entities: Physical Line(“p_side”) and Physical Line(“n_side”).

```
Physical Line("p_side") = {1}
Physical Line("n_side") = {2}
```

In this way, these physical regions are made available for **TiberCAD** , and will be associated to a **TiberCAD Region** (see in the following).

Then, two **Physical Points** are defined, **anode** and **cathode** , and associated to the first and to the last point of our 1D device.

```
Physical Point("anode") = {1}
Physical Point("cathode") = {3}
```

These points are needed to impose some boundary conditions and in this way they are made available for **TiberCAD** , and will be associated each to a boundary condition region (see in the following).

A *non-interactive* mode is also available in GMSH, without graphical user interface. For example, to mesh this first 1D tutorial in non-interactive mode, just type:



```
gmsht diode.geo -1 -o diode.msh
```

where `diode.geo` is the geometrical description of the device with GMSH syntax;

some command line options are:

- -1 , -2, -3 to perform 1D, 2D or 3D mesh generation
- -o **mesh_file.msh** to specify the name of the mesh file to be generated

2.2.2 TiberCAD input file

This is the summary of the Sections of this Tutorial :

- *Device structure*

- *Simulation models*
- *Solver parameters*
- *Physics parameters*
- *Run simulations*
- *Output*

2.2.3 Device structure

Let's have a look to the input file; for further details you can refer to the reference manual.

```
Device
{
  meshfile = diode_1D.msh
  material = Si

  Region n_side
  {
    Doping
    {
      Nd = 1e18
      type = donor
    }
  }

  Region p_side
  {
    Doping
    {
      Nd = 1e18
      type = acceptor
    }
  }
}
```

First we define the device structure: it is composed by two Regions, both constituted of the material Silicon, respectively n and p doped with a concentration of $1e18 \text{ cm}^{-3}$.

With Region n_side

and Region p_side

the physical regions previously defined in GMSH are associated to the respective **TiberCAD Region** .

2.2.4 Simulation models

Now, we have to define the **simulations** which we are going to execute in our calculations.

One **simulation** is a particular set of equations, boundary conditions, physical parameters, solver parameters, which describes a physical problem to be solved by TiberCAD.

A valid **TiberCAD simulation** must belong to one of the predefined **TiberCAD simulation models**.

TiberCAD, being a **multiphysics** software tool, includes several simulation models, each one describing a physical problem to be solved, e.g DriftDiffusion (to solve Poisson and DriftDiffusion equations) , EFASchroedinger (to solve Schroedinger equation in effective mass approximation) , Macrostrain (to calculate macroscopical strain) and others.

To create a **TiberCAD simulation** , we first have to declare the **TiberCAD model** class to which our simulation belongs:

```
Module driftdiffusion
{
    name = dd

    plot = (Ec, Ev, Eg, ElField, ElPotential, eQFermi, hQFermi, eDensity, hDensity,
            NetRecombination, eCurrentDensity, hCurrentDensity, CurrentDensity,
            ContactCurrent, eMobility, hMobility, IonizedDonors, IonizedAcceptors)
```

Here, we declare that the simulation to be created will belong to the model class **driftdiffusion** (`model driftdiffusion`)

In the next Options block, we define the name of the TiberCAD simulation (`name = dd`) and the TiberCAD regions to which it will be applied (`region = Regiona_Name` if you don't specify anything, it means the whole device).

After we specify types of data we want in our Output file.

In this example, just one simulation for the *driftdiffusion model* is created.

In general, several TiberCAD simulations belonging to the same **model** can be created; each of them must have a different **name** . This name must also be different from the **model name**.

As we will see in the following, it is possible to specify physical and solver parameters for a single **TiberCAD simulation** , by referring to its **name** .

Anyway, it is also possible to specify parameters common to all the **TiberCAD simulations** belonging to the same **model** , by referring to the name of the **TiberCAD model** instead (in this example driftdiffusion):

```
Physics
{
    particle_density
    {
```

```

        statistics = boltzmann
    }

    # use a field dependent mobility model with the
    # gradient of the electrochem. potential as field
    # parameter
    mobility field_dependent
    {
        low_field_model = doping_dependent
    }
    # use SRH and Auger recombination
    recombination (srh, auger) {}
}

Contact anode
{
    type = ohmic
    voltage = $Vb[0.0]
}

Contact cathode
{
    type = ohmic
    voltage = 0.0
}

}

```

Thus, in this simulation we are going to calculate *Poisson* and *Drift-diffusion* (**model driftdiffusion**) for all the device (**regions = all** if not specified).

Next, some `physical_model` blocks follow. The `physical_model` blocks are used in general to define the physical model to be used to describe a physical property or quantity associated to the present TiberCAD model.

Here, for example, we choose a Schottky-Read-Hall model (`model = srh`) to describe recombination in our driftdiffusion calculations.

`physical_model recombination`

Physics

```

    { recombination (srh, auger) {}
    }

```

Also, we choose a field dependent model to describe electron and hole mobility (`mobility = field_dependent`), with the low field mobility determined by a doping-dependent model (Masetti model, see reference manual) (`low_field_model = doping_dependent`).

```
physical_model electron_mobility { model = field_dependent low_field_model = dop-
ing_dependent }
```

Now, we have to specify the Boundary conditions for the PDE problem represented by the simulation `dd` : they are defined in `Contact` section.

Here, two boundary conditions are defined, corresponding to the two contacts of our pn diode, anode and cathode.

```
Contact anode
{
    type = ohmic
    voltage = $Vb[0.0]
}
```

```
Contact cathode
{
    type = ohmic
    voltage = 0.0
}
```

Here the **anode** and **cathode** Contacts are associated to the corresponding anode and cathode regions defined in GMSH as Boundary condition regions (which in the present case of 1D simulation are actually just a point), with

- *Contact anode*

and

- *Contact cathode* .

Besides, the type of Boundary condition is assigned (in this case **ohmic**) with its (initial) value, given by **voltage** .



The anode voltage is expressed by the notation **@Vb[0.0]**. This means that the anode voltage will be given the value of the variable **Vb** , specified in the **sweep** block (see after). **[0.0]** means that the default voltage value is 0.0 V.

2.2.5 Solver parameters

Definition of Model-dependent Solver parameters

```
Module sweep {
    variable = $Vb solve = dd
    start = 0.0 stop = 1.2 steps = 60
    plot_data = true # to write data for each step
}
```

```
}
```

Now, the parameters for the **solver** are defined.

These values are tuned for the specific simulation and usually don't need to be changed; however you can refer to the manual for further details.

The **sweep** block allows us to perform the calculation of the IV characteristic of the diode: it defines the variable V_b , which will assume a range of values between 0 and 1.2 V. These values will be assigned in turn to the *anode* contact, in order to execute a series of complete dd simulations (simulation = dd) for each bias condition.

2.2.6 Physics parameters

Definition of Model dependent physical parameters:

```
Physics
{
  particle_density
  {
    statistics = boltzmann
  }
  mobility field_dependent
  {
    low_field_model = doping_dependent
  }
  recombination (srh, auger) {}
}
```

In the Physics section usually some physical parameters are defined. In this case we state that the Fermi-Dirac statistic will be applied (statistics = FD).

2.2.7 Run simulations

Finally, in Simulation section, we define the actual simulation to be performed, specified by:

```
solve = sweep
```

this determines the execution of dd inside the sweep cycle; this means that we are going to perform a calculation of **Poisson** and **drift-diffusion** for all the biases defined in the sweep section.

Results of the calculation will be in the format of xmgr data visualization program (output_format = grace), suited to 1D case, since it consists of ascii text files with column data.

```
Simulation
{
```

```
verbose = 2

solve = sweep

resultpath = diode1D_results
output_format = grace
}
```

Note that the output variable `ContactCurrents` is needed to get in output the IV characteristic.

Now we can run TiberCAD....

```
tibercad diode.tib
```

During the execution, the screen output shows some information about how the simulation is going:

First, about the creation of the Device regions; if there is an inconsistency between the mesh file and input file device description, here an error message is displayed and the execution terminated.

If all is ok, the calculation is started:

```
We solve: sweep
```

In this example only one simulation is performed, sweep, which is a set of drift-diffusion calculations.

First, the equilibrium solution is found, by solving Poisson equation:

```
DriftDiffusion (name:  driftdiffusion) Solving
equilibrium ..... iterations:  6, residual =
1.95054e-16 Equilibrium done
```

Then, current continuity equations are solved together with Poisson eq., for each step of the sweep.

The voltage and current found for each contact is displayed:

```
Sweep value Vb = 0.05
```

```
<<-----
```

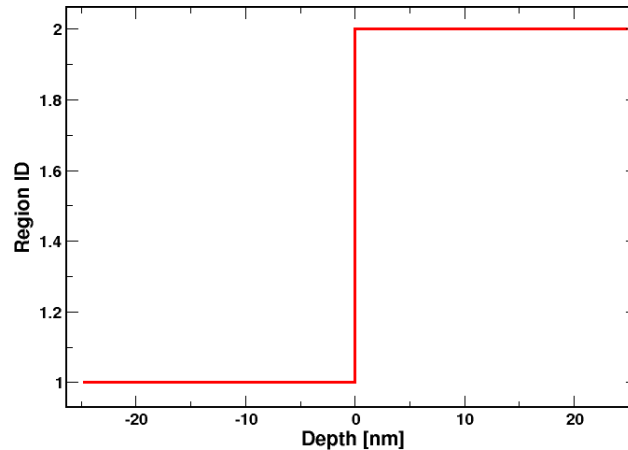
```
iterations: 5, residual = 1.17131e-10
```

```
contact name: contact voltage: contact current: cathode 0 6.31825e-05 anode 0.05
-6.3187e-05''
```

The simulation ends correctly (hopefully...) with the Sweep value $V_b = 1.2$.

2.2.8 Output

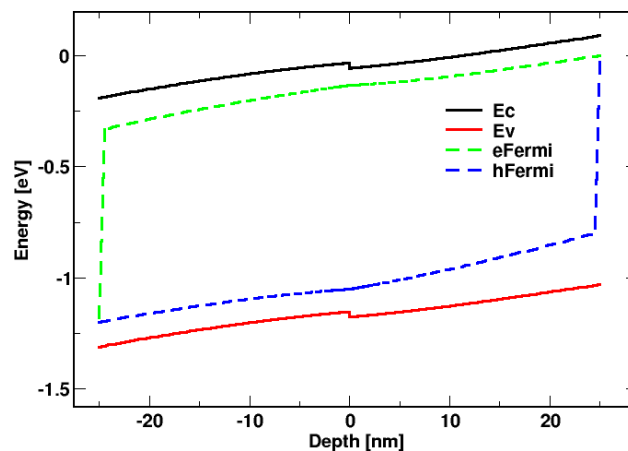
Now, the output directory contain the simulation results, as defined in `output_format` and `plot`.



Thus, in `driftdiffusion_materials.dat` we have the information about the material regions of the device (not very interesting in 1D, but useful in 2 and 3D).

In `driftdiffusion_nodal_Vb_0.000.dat` and all the other files for each bias step, we have the output for the nodal quantities which have been calculated, e.g. conduction and valence bands, (quasi)fermi levels, electron and hole density and mobility.

Here is, for example, the band profile obtained at the last step of calculation, for $V = 1.2V$.



The IV characteristic of the diode is contained in the file `sweep_driftdiffusion_Vb.dat`, where the currents at both the contacts are reported.

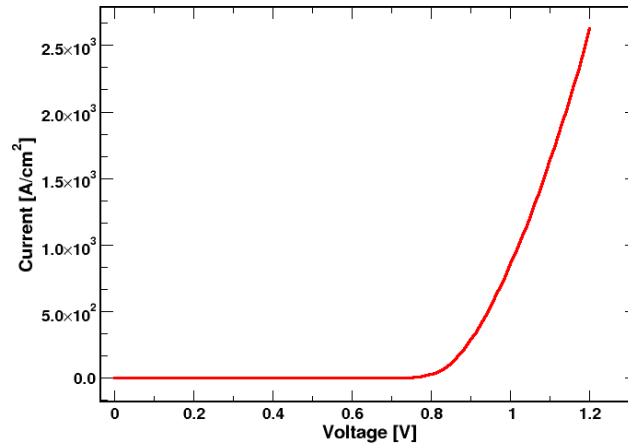
In the file `driftdiffusion_elemental_Vb_0.200.dat` etc., you can find the elemental quantities which have been calculated, in this case the electron, hole and total current density (J_n , J_p , J_x).

Attachment Size

[diode_1D.tib](#) 1.79 KB

[diode_1D.geo](#) 239 bytes

[diode_1D.msh](#) 26.24 KB



2.3 n-MosFET 2D

This is the summary of the Sections of this Tutorial :

- *Device structure*
- *Drift-diffusion simulation*
- *Calculation of IV drain characteristic*
- *Physic*
- *Output*
- *Calculation of transfer characteristic*

In this tutorial we will show an example of **2D** simulation of a **Si n-MOSFET** device.

We will calculate :

- IV drain characteristic,
- Id/Vg transfer characteristic.

In order to execute correctly the example you should have the following files in the working directory:

`mosfet.tib` : input file for TiberCAD `mosfet.msh` : mesh file produced by GMSH from the script `mosfet.geo`

Here is the geometry of the Mosfet device as it is meshed by GMSH.

Let's give now a look to the input file; for further details you can refer to the program reference manual.

2.3.1 Device structure

Description of the device physical regions:

```
Device mosfet
{
    meshfile = mosfet.msh
    material = Si

    Region substrate
    {
        Doping
        {
            Nd = 1e18
            type = acceptor
        }
    }

    Region contact
    {
        Doping
        {
            Nd = 5e19
            type = donor
        }
    }

    Region oxide
    {
        material = SiO2
    }
}
```

First we define the device structure: it is composed by three **TiberCAD Regions**: **substrate** and **contact** , both made of *Silicon* , and *oxide* (SiO2).

The **substrate** region (which include channel) is doped p-type $10^{18}cm^{-3}$; contact region is composed by the two regions around source and drain, both heavily doped n-type $5 \times 10^{19}cm^{-3}$; finally, **oxide** is the gate dielectric.

2.3.2 Drift-diffusion simulation

Now, we define the **TiberCAD simulation models** which will be used in this example. We are going to calculate Poisson and Drift-diffusion (model *driftdiffusion*) for all the device (regions = all in options).

A field dependent model for **mobility** is applied.

```
recombination srh {}

mobility
{
  type = field_dependent
  low_field_model = doping_dependent
}
```

Then, we have to specify the the three **contacts** of our Mosfet.

We assume a gate width of 1 mm.

```
Contact gate
{
  type = schottky
  barrier_height = 3.0
  voltage = $Vg
  area_factor = 0.1
}
```

```
Contact source
{
  voltage = 0.0
  area_factor = 0.1
}
```

```
Contact backcontact
{
  voltage = 0.0
  area_factor = 0.1
}
```

```
Contact drain
{
  voltage = $Vd
  area_factor = 0.1
}
```

The contacts (boundary regions for this model) have to be associated each to the corresponding region defined in the mesher program (here GMSH) as a Boundary region (which in the present case of 2D simulation is a line (`Physical Line` in GMSH)).

This is made simply with

```
Contact gate
```

and so on for the other contacts.

The **gate** contact is defined as a **schottky** contact, with a barrier value depending on the gate metal

`workfunction ( barrier_height )`.

The gate voltage is expressed by the notation `@Vg[0.0]` . This means that the gate voltage will be given the value of the variable `Vg` , specified in one of the sweep block (see after). **[0.0]** means that the default voltage value is 0.0 V. Source and drain contacts are defined as ohmic; while source is fixed to 0 (`voltage = 0.0`), drain voltage is expressed, as for the gate, by the value of the sweep variable `Vd` (`voltage = @Vd[0.5]`).

2.3.3 Calculation of IV drain characteristic

Definition of Model-dependent Solver parameters:

```
Module sweep
{
  name = sweep_drain
  solve = driftdiffusion

  variable = $Vd
  start = 0.0 # 0.2
  stop = 1.0
  steps = 5 # 4
}
```

```
Module sweep
{
  name = sweep_gate
  solve = driftdiffusion

  variable = $Vg
  start = -0.5
  stop = 1.5
  steps = 100

  max_step = 0.1
  plot_data = true
}
```

Every cycle is performed at maximum step of 0.1 V

Definition of model-independent parameters of the Simulation:

```
Simulation
{
  temperature = 300
  solve = (sweep_drain, sweep_gate)
  resultpath = output_transchar
}
```

```
output_format = vtk
}
```

Now, the parameters for the solver are defined, for the **driftdiffusion model**. `coupling = electrons` specifies that only one type of carrier is considered.

`ksp_type = bcgsl` defines the solver type,

`nonlinear_solver = tiber` specifies the non-linear solver.

As a first step, we are going to calculate **Id/Vd drain current characteristic** .

We include the optional **Sweep** block, whose syntax is the following:

```
Module Sweep
{
  name = sweep_name1
  {
    .....
  }
}
Module Sweep
{
  name = sweep_name2
  {
    .....
  }
}
```

Here we define two nested **sweeps** : the external sweep (`sweep_gate`) calculates the drain current for a series of gate voltages, from -0.1 to 0.5 V; for each calculation, a sweep on drain voltages is performed (`simulation = sweep_drain`), defined by `sweep_drain` . In `sweep_drain`, `driftdiffusion` is calculated (`simulation = driftdiffusion`) for a series of drain voltages from 0 to 2 V.

2.3.4 Physic

In the **Physics** section usually some physical parameters are defined. In this case we just state that we want to use the **unstrained** model for `driftdiffusion` (since no strain calculation is intended) and that we use Fermi-Dirac statistic

```
Physics {
  particle_density {
    statistics = fermidirac
  }
  recombination srh {}
}
```

```

mobility {
    type = field_dependent low_field_model = doping_dependent
}

```

Finally, in **Simulation** section, we define the 2D (`dimension = 2`) simulation to be performed, specified by

`solve = sweep_gate` : this means that we are going to perform a calculation

of **Poisson** and **drift-diffusion** with a double sweep on gate and drain voltage, to calculate **Id/Vd drain current characteristics** .

Results of calculation will be in the format of **paraview** data visualization and post-processing program (`output_format = vtk`)

In **plot** we define the output variables to be calculated.

```

Simulation
{
    temperature = 300

    solve = sweep_gate
    resultpath = output_outputchar

    output_format = vtk
}

```

Now we can run TiberCAD....

```
tibercad mosfet.tib
```

2.3.5 Output

After the execution, the output directory contain the simulation results, as defined in `output_format` and `plot`.

TiberCAD s ports 3 packages for 2D and 3D data visualization and post-processing:

Free:

1. GMV, <http://www-xdiv.lanl.gov/XCM/gmv/GMVHome.html>

```
output_format = gmv
```

2. Paraview, <http://www.paraview.org/New/index.html>

```
output_format = vtk
```

Commercial:

1. Tecplot-ise, <http://www.amtec.com/>

```
output_format = ise
```

Let's see how to use Paraview to plot TiberCAD 2D results:

First, open the .vtk file from your directory:

The name of the loaded files will be shown in the Pipeline browser.

To visualize the content of the file, you should click on the Apply button in the **Properties** tab in **Object Inspector**

To select the output variable , go to **Display** and choose from the menu “Color by”, e.g. electron _density. Also, in Display section Scale and legend bar can be setted.

Thus, in driftdiffusion_materials.vtk we have the information about the material regions of the device: here, **1** indicates the substrate Region , **2** the contacts region and 3 the SiO2 gate dielectric.

In driftdiffusion_nodal.vtk and all the other files for each bias step, we have the output for the nodal quantities which have been calculated, e.g. conduction and valence bands, (quasi)fermi levels, electron and hole density and mobility.

For example, this is the electron density for a drain voltage = 2V and a gate bias $V_g = 1V$. You can see the large increase of electron density in the channel, which is pinched off close to drain contact due to the high drain voltage.

In sweep_2_driftdiffusion_Vg_0.000_Vd.dat and all the other files for each Vg bias step (sweep_2_driftdiffusion_Vg_step_Vd.dat) we have the IV drain characteristics for each Vg bias.

Here are the IV characteristics obtained for a gate voltage Vg between -0.1 and 0,5 V.

2.3.6 Calculation of transfer characteristic

As a second step, we are going to calculate Id/Vg transfer characteristic. (see input file [mosfet_transchar.tib](#))

To do so, the two **sweeps** have to be defined in this way:

```
Module sweep
{
  name = sweep_drain
  solve = driftdiffusion

  variable = $Vd
  start = 0.0
  stop = 2.0
```



```

        steps = 40

        plot_data = true
    }

Module sweep
{
    name = sweep_gate
    solve = sweep_drain

    variable = $Vg
    start = 0.0
    stop = 1.5
    steps = 6
}

```

the first sweep (sweep_drain) calculates the drain current (simulation = driftdiffusion) for a series of drain voltages, from 0 to 1 V, while Vg is kept at its initial value Vg=0; at the end of this calculation we have polarized the mosfet at a drain voltage of 1 V.

Then a sweep on gate voltages is performed, defined by sweep_gate . In sweep_gate , driftdiffusion is calculated (simulation = driftdiffusion) for a series of gate voltages from -1 to 0.5 V, by keeping the drain voltage at Vd = 1 V.

At the end, the Id/Vg transfer characteristic for Vd=1 V is obtained.

This time, in **Simulation** section, we define the 2D simulation to be performed by writing solve=(sweep_drain, sweep_gate) : this means that we are going to perform a calculation of **Poisson** and **drift-diffusion** with two sweeps in series, first a sweep on drain and then a sweep on gate voltage, to calculate **Id/Vg transfer characteristic** .

```

Simulation
{
    temperature = 300
    solve = (sweep_drain, sweep_gate)
    resultpath = output_transchar

    output_format = vtk
}

```

The rest of the input file remains unchanged.

Now we can run TiberCAD another time....

```

tiberCAD mosfet.tib

```

- Now, in the files sweep_1_driftdiffusion_Vd_step_Vg.dat (e.g. sweep_1_driftdiffusion_Vd_1.000_Vg.dat), we have the Id/Vg transfer

characteristics for each V_{d_step} bias.

Here is the **Id/Vg transfer characteristic** for a drain voltage $V_d = 1\text{ V}$.

As before, in `driftdiffusion_elemental.vtk` and `driftdiffusion_nodal.vtk`, we have the output for respectively the nodal and the elemental quantities which have been calculated.

The **subthreshold S** parameter, given by
in this case results to be **~80 mV/dec**.

Attachment Size

`mosfet.msh` 309.63 KB
`mosfet.geo` 1.67 KB
`mosfet.tib` 2.28 KB
`mosfet_transchar.tib` 2.14 KB

2.4 Bjt1 & Bjt2

This is the example of the first Bjt.

2.4.1 Device

```
Device bjt
{
  material = Si
  meshfile = bjt.msh

  Region coll_doped # the buried doping
  {
    Doping
    {
      type = donor
      density = 1e19
    }
  }

  Region coll # the collector region
```

```
{
  Doping
    {
      type = donor
      Nd = 5e15
    }
}

Region b_reg # the base region
{
  Doping
    {
      type = acceptor
      density = 1e17
    }
}

Region e_reg # the emitter region
{
  Doping
    {
      type = donor
      density = 1e19
    }
}
}
```

2.4.2 Module

```
Module driftdiffusion
{

  name = dd

  # we use a different quadrature rule to avoid density peaks
  # where the mesh is not so dense
  quadrature_rule = trapez

  plot = (Ec, Ev, ElPotential, ElField, eQFermi, hQFermi, eDensity, hDensity,
          eCurrentDensity, hCurrentDensity, CurrentDensity, ContactCurrents,
          NetRecombination, eMobility, hMobility)

  Solver linesearch
```

```
{  
}
```

Physic

Physics

```
{  
  
    # use Fermi-Dirac statistics  
    statistics = FD  
  
    mobility doping_dependent {}  
  
    recombination (srh, auger) {}  
}
```

2.4.3 Contacts

Contact base

```
{  
    type = ohmic  
    voltage = $Vb  
}
```

Contact collector

```
{  
    type = ohmic  
    voltage = $Vc  
}
```

Contact emitter

```
{  
    type = ohmic  
    voltage = 0.0  
}
```

2.4.4 Sweep

Module sweep

```
{  
    name = sweep_base  
    variable = $Vb  
}
```

```
start = 0.5
stop = 0.7
steps = 10

# we calculate the output characteristics
solve = sweep_coll

# plot only for the last collector bias
plot_data = true
}

Module sweep
{
  name = sweep_coll
  variable = $Vc
  start = 0.0
  stop = 3
  steps = 60

  solve = dd
  #plot_data = true
}
```

2.4.5 Simulation

The temperature = 300 is the default

```
Simulation
{
  verbose = 3

  solve = sweep_base

  resultpath = output
  output_format = vtk
}
```

2.4.6 Device

This is the example of the second bjt

```
Device bjt2
{
```

```
material = Si
meshfile = bjt2.msh

Region coll_doped # the buried doping
{
  Doping donor { density = 1e19 }
}

Region coll # the collector region
{
  Doping
  {
    type = donor
    Nd = 5e15
  }
}

Region b_reg # the base region
{
  Doping
  {
    type = acceptor
    density = 1e17
  }
}

Region e_reg # the emitter region
{
  Doping
  {
    type = donor
    density = 1e19
  }
}

Region oxide
{
  material = SiO2
}
}
```

2.4.7 Module

```
Module driftdiffusion
{
```

```
name = dd

quadrature_rule = trapez

plot = (Ec, Ev, ElPotential, ElField, eQFermi, hQFermi, eDensity, hDensity,
        eCurrentDensity, hCurrentDensity, CurrentDensity, ContactCurrents,
        NetRecombination, eMobility, hMobility,
        IonizedDonors, IonizedAcceptors,
        IonizedElectronTraps)

Solver linesearch
{
}
```

2.4.8 Physic

We'll use the Fermi-Dirac statistic:

```
Physics
{

    statistics = FD

    mobility doping_dependent {}

    recombination (srh, auger) {}

    trap eNeutral
    {
        regions = Si/SiO2
        Nt = 2e12
        Et = 0.5
        reference = cb
    }
}
```

Contacts

```
Contact base
{
    type = ohmic
    voltage = $Vb
}
```

```
Contact collector
{
  type = ohmic
  voltage = $Vc
}
```

```
Contact emitter
{
  type = ohmic
  voltage = 0.0
}
```

2.4.9 Sweep

We can calculate the output characteristics and plot only for the last collector bias:

```
Module sweep
{
  name = sweep_base
  variable = $Vb
  start = 0.5
  stop = 0.7
  steps = 10

  solve = sweep_coll

  plot_data = true
}
```

```
Module sweep
{
  name = sweep_coll
  variable = $Vc
  start = 0.0
  stop = 3
  steps = 60

  solve = dd
  plot_data = true
}
```

2.4.10 Simulation

The temperature = 300 is the default


```
Simulation
{
  verbose = 3

  solve = sweep_base

  resultpath = output2
  output_format = vtk
}
```


THERMAL

3.1 Boundary conditions

The Dirichlet boundary condition is set with

```
Contact reservoir
{
    type = heat_reservoir
    temperature = 300
}
```

whereas the surface thermal resistance is applied with

```
Contact dissipator
{
    r_surf = 0.5
    type = surface_resistance
    temperature = 300
}
```

The selfconsistent simulation solves the thermal and the drift diffusion simulations (named **tt** and **dd** respectively) in a self consistent way.

```
Module Selfconsistent
{
    flavour = relaxation
    simulations = (dd,tt)
    abs_tolerance = 1e-5
    monitor = true
}
```

The sweep section runs the selfconsistent simulation for each anode bias step

```
Module sweep
{
```

```
simulation = selfconsistent
variable = Vb
start = 0.0
stop = 1.0
steps = 10
}
Simulation
{
  temperature = 300
  solve = sweep
  symmetry = cylindrical
}
```

The temperature map, the heat source and the thermoelectric power are shown below.

```
Contact heat_reservoir
{
  region = substrate
  temperature = 300
}
```

```
Contact substrate
{
  type = surface_resistance
  r_surf = 0.05
  temperature = 300
}
```

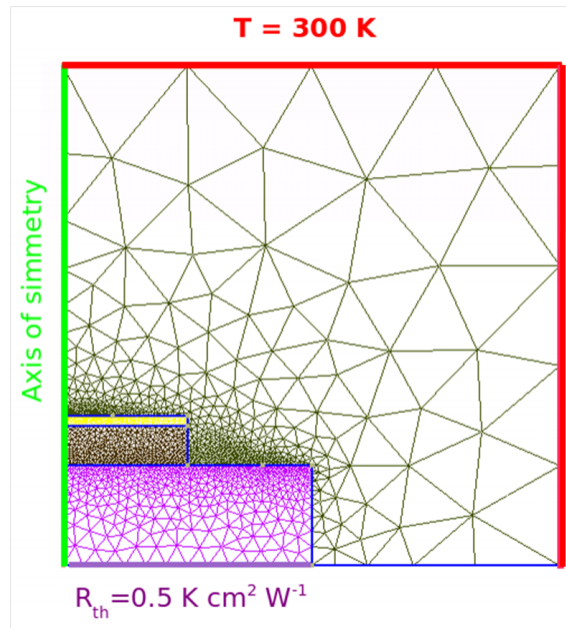
3.2 Heat sources

In this example we will see how to perform a thermal-drift-diffusion self-consistent simulation. We include the Seebeck and Peltier effects. The heat conduction through the environment is modelled by adding an air region all around the diode. We assume that far from the junction the system is in thermal equilibrium with a thermal bath at 300 K.

The heat dissipation through the substrate is taken into account by introducing a thermal surface resistance. Furthermore, we rely on the cylindrical symmetry with respect to the axis of growth and consider only a 2D slice of the system. In this way we simulate a 3D device by performing a 2D simulation (with much less computational time). The mesh corresponding to the whole system (diode plus air region) is shown below.

Region n-side corresponds to the n-doped region situated at the bottom of the device, while the region p-side refers to the top p-doped region. Air is modelled with Region Air.

```
Device
{
```



```

meshfile = tut_09.msh
dimension = 2
material = Si
Region n_side
{
  doping = 1e18
  doping_type = donor
}
Region p_side
{
  doping = 1e19
  doping_type = acceptor
}
Region air
{
}
}

```

The drift diffusion simulation is performed only over the device domain (excluding the Air region). Therefore, in option subsection we have to indicate *physical_regions = (n side, p side)* . The mobility models used for both electrons and holes are doping dependent.

```

electron_mobility doping_dependent{}

hole_mobility doping_dependent{}

```

The connection with the thermal simulation is given by the keyword *thermal_simulation = tt* . The thermoelectric power model used can be included with *thermoelectric diffusivity model* (see

the **Theory** section). Direct bias applied to the diode is obtained by using the following boundary conditions

```
Contact anode{@Vb[0.0]}
```

```
Contact cathode{}
```

Unlike the drift diffusion case, the thermal simulation has to be performed over the whole simulation domain (including the Air region). We can refer to all regions simply by writing *physical_regions = all* . The connection with the drift diffusion simulation is specified with

```
HeatSource joule
{
    DD_simulation_name = dd
}
```

```
HeatSource constant
{
    region = qdot
    H = 1e10
}
```

INPUT FOR TIBERCAD

Input for TIBERCAD is composed by an input file e.g. “input.tib” and a mesh file generated by a mesher software: as for now, mesh files from GMSH (*.msh*, *v.1 and v.2.0*) and from ISE-TCAD (*.grd*) are supported. Be sure that the material files are in the correct directory . To run the program, type: **tibercad** *input_file_name*

4.1 Description of Input file structure

A valid input file for TIBERCAD is a text file with the structure described in the following. In the whole input file, everything following a “#” is considered as a comment and is disregarded; blank lines can be present anywhere and are disregarded too. Input file is composed by several **blocks** : A block is enclosed between “{” and “}” brackets and may include one or more blocks. Each block has a header made of one or two keywords. Each block may contain zero or any number of **parameter assignments** in the form:

” *tagname* = *tagvalue*” , where

- “*tagname*” is a string
- “*tagvalue*” is a single numerical or string item or a list of items between “(” and “)”

parenthesis and separated by commas. e.g. (*cathode*, *anode*)

Format is free for the parameter assignments, provided that they are separated by spaces. Everything which follows a “#” is considered as a comment and is disregarded. For example:

```
electron_mobility field_dependent
{
  region = buffer
  # type = doping_dependent
  low_field_model = constant
}
```

Here and in the whole input file a string item can include a combination of characters, special characters and numbers, except spaces; if a space is found, the string item is taken as terminated.

The input file is composed by the following main classes of blocks:

Device, Module, Simulation

which will be described in the following.

4.2 Device section

```
Device hemt1
{
  meshfile = hemt_msh.grd
  Region buffer
  {
    .....
    Doping
    {
      .....
    }
  }
  Region barrier_1
  {
    .....
  }
  .....
}
```

Device section includes the geometrical description of the device to be simulated; an optional *device name* can be associated to the Device object, e.g. *Device hemt1* .

In Device section, two kinds of block can be present: the Region block and the Cluster block. Outside of these blocks, general options common to all the device can be defined. The most important is the mesh file definition:

meshfile : name of mesh file. N.B.: the extension is mandatory! (*.grd* for ISE-TCAD, *.msh* for GMSH mesh file v.1 and v.2.0)

The definition of the mesh file associated to this simulation is compulsory:

```
meshfile = hemt_msh.grd
```

The **Region** blocks contain the description of the device in continuous media approach; the **Cluster** blocks define each a group of regions (mesh regions) even with different physical properties, but to be treated together somewhere in the simulation (e.g. quantum calculation). In this way it is possible to refer to the set of these regions simply by the **Cluster** name.

Each **Region** block must be preceded by the keyword “**Region**” , followed by the (single-word) name of the **TiberCAD Region** . The name of the **TiberCAD Region** can coincide with the

name of a mesh region, as defined during the modeling of the device; in this case, if the keyword *mesh_regions* is absent, the **TiberCAD Region** will be associated to the mesh region identified by the name assigned to the **TiberCAD Region** . Otherwise, the **TiberCAD Region** will be associated to the mesh regions specified by the keyword *mesh_regions* .

```
Region QWell
{
  mesh_regions = (weell1, well12)
  structure = wz
  y-growth-direction = (1, 0, -1, 0)
  z-growth-direction = (-1, 2, -1, 0)
  x-growth-direction = (0, 0, 0, 1)
  material = GaN
  Doping
  {
    density = 1e17
    type = donor
    level = 0.025
  }
}
```

Here are the description of the available keywords for a **Region** block.

material (mandatory): name of the material associated to the present region, e.g Si;

it may be a ternary alloy, e.g AlGaAs, in this case keyword x described in the following has to be present.

x : alloy concentration, expressed as the molar fraction of the first component of

the alloy; e.g. to express an alloy $Al_xGa_{1-x}As$ with molar fraction $x = 0.2$, that is

$Al_{0.2}Ga_{0.8}As$, we select **AlGaAs** for the keyword material, and 0.2 for the keyword x,

thus we write $x = 0.2$.

mesh_regions : (a list of) region physical name(s) as specified in the meshing program.

structure : crystal structure (wz = wurtzite, zb = zincblend)

x-growth-direction, *y-growth-direction*,
z-growth-direction : Bravais vectors with Miller

indexes for wurtzite crystal (4 element vectors) or zincblende crystal (3 element vectors).



A common crystal structure and growth directions definition may be applied to all the Regions of the Device, just by defining them everywhere in the Device block, but outside any specific Region block.

Subblock doping, with the keywords:

density : doping concentration [cm-3] type : donor or acceptor level : energy level of the dopant [eV]

Doping

```
{
  density = 1e21
  type = donor
  level = 0.026
}
```

Each **Cluster** block must be preceded by the keyword “**Cluster**” , followed by the (single-word) name of the **Cluster** .

```
Cluster Quantum_1
{
  regions = (well, barrier1,barrier2)
}
```

regions (mandatory): list of physical regions as specified in the meshing program, or TIBERCAD region names to be grouped in the cluster.

Regions and **Clusters** represent the macroscopical description of the device or structure to be simulated in **TiberCAD** . In the rest of the input file, the physical regions associated to the **Modules** will be indicated by means of the **TiberCAD Region** and

Cluster names.

4.3 Modules

Module driftdiffusion

```
{
  name = dd
  #regions = all
  statistics = FermiDirac
  strain_simulation = macrostrain
  plot = (Ec, Ev, Eg, eQFermi, hQFermi, eDensity, hDensity, Polarization,
  eCurrentDensity, hCurrentDensity, CurrentDensity, ContactCurrents,
  ElPotential, ElField, eMobility, hMobility, eConductivity, hConductivity)
  .....
  Solver
  {
    nonlinear_solver = tiber
    nonlin_max_it = 15
    nonlin_rel_tol = 1e-12
    #nonlin_abs_tol = 1e-15
```

```

        nonlin_step_tol = 1e-5
    }
Physics
{
    polarization (piezo, pyro) {}
    mobility
    {
        type = field_dependent
        low_field_model = doping_dependent
    }
}
Contact cathode
{
    voltage = $Vd
}
Contact anode
{
    voltage = 0.0
}
}

```

One or more module-blocks may be present: each module-block must be preceded by the keyword **“Module”** , followed by the (single-word) module name. This must be the name of one of the TIBERCAD modules. Here are the Modules implemented until now:

- `driftdiffusion` : Poisson-driftdiffusion transport of electrons and holes
- `thermal` : Heat balance simulation
- `excitontransport` : Exciton transport model
- `macrostrain` : Calculation of Elastic deformations in heterostructures
- `efaschroedinger` : Envelop Function Approximation (EFA) solution of single particle

Schroedinger equation for electrons and holes

- `quantumdensity` : Calculation of quantum density of electrons and holes.
- `quantumdispersion` : Dispersion of quantized states in k space
- `opticskp` : Optical properties (optical kp matrix elements)
- `opticalspectrum` : Emission spectrum (with k-space integration)
- `Sweep` : Parameterized execution of a module simulation (e.g. for the calculation

of output current characteristics)

- `Selfconsistent` : coupled calculations of different simulation modules.
- `DSC` : Simulation of a DSC solar cell1.

Each module-block usually contains a list of general options, such as plot and others specific to each module. Then, two main blocks define the Physics and the Solver models and parameters for this module.

Solver contains the solver parameters; depending on the Module, it can contain a LinearSolver definition subblock.

Physics usually contains the definition of the physical models used in the simulation.

For example, for driftdiffusion module, recombination, electron mobility, trap, polarization and so on. A particular model is the Boundary model, which has an alias Contact for driftdiffusion module. The declaration of these models obey to the following syntax:

model_keyword type_specifier <block>

where *model_keyword* is the name of the physical model to be declared (e.g. trap model) and **type_specifier** is the name of a particular one among the available descriptions for that model. For example, for a trap model of **type** *acceptor*

```
trap acceptor
{
  region = buffer
  Nt = 7e16
  Et = 0.5
  reference = cb
}
```

An alternative multiple declaration is possible if no parameters, beyond default, are declared:

model_keyword (type specifier1, type specifier2,...)

```
recombination (srh, direct, auger) { }
```

In this example, several recombination models are defined (srh, auger, direct) each one with default parameters. For a detailed description of the models, please refer to the reference guide. Two special Modules are: the Sweep Module and the Selfconsistent Module

4.3.1 Module sweep

```
Module sweep
{
  name = sweep_drain
  solve = driftdiffusion
  variable = $Vd
  start = 0.0
  stop = 2.0
  steps = 20
  plot_data = true
}
```

```

Module sweep
{
  name = sweep_gate
  solve = sweep_drain
  variable = $Vg
  start = -0.1
  stop = 1.0
  steps = 11
}

```

Each sweep Module defines a set of calculations applied to a boundary region (e.g. a set of bias values to be assigned to a drain contact of a MOSFET for the calculation of an output drain IV characteristic), in this Guide referred to as sweep calculation.

The following keywords are defined for this feature:

`variable` : name of the variable to which the sweep is applied:

E.g.: `variable = $Vg`

indicates that the values are applied to the variable Vg, which is a quantity defined in a Contact definition

`start, stop, steps` : sweep starts from start value, is repeated steps times and stops

in stop

`solve` : name of the simulation (module) associated to the sweep calculation; it may

be the name of another sweep defined in the same block.

`plotvariable` (obsolete): specify the integrated quantity to be calculated during the

sweep and that will be shown in the output file `sweep modelname sweepvariable.dat`, eg. `sweep driftdiffusion Vb.dat` for a sweep of current calculation on the variable Vb (typically a contact voltage).

`plot_data` : default is false; if it is set to true, then output data will be written for

each step of the sweep calculation, otherwise just the results for the final step will be present in the output.

Once a *sweep* calculation has been defined, it is treated as a special case of simulation and may be executed as an usual simulation: by adding it in the solve list, e.g. `solve = sweep_drain`.

4.3.2 Module selfconsistent

In this Module it is possible to define a self-consistent calculation based on two different simulation modules (e.g. `driftdiffusion` and `excitontransport`).

```
Module selfconsistent
{
  name = sc_all
  solve = (tb, dens_el, dens_hl, dd)
  max_iterations = 5
  abs_tolerance = 1e-3
  rel_tolerance = 1e-6
}
```

In **solve** the list of simulations to be performed self-consistently is specified. Now it is possible to execute the specified simulations in self consistent way, by using the **selfconsistent** keyword like a simulation name, in any **solve** assignement, e.g. solve = selfconsistent in **Simulation** section, or even in a **sweep** section.

4.4 Simulation section

In this block one can specify several general parameters and settings for the actual calculation to be run, such as the temperature, the process-flow of simulation, etc.

searchpath : path for material files
mesh units : units of measurements used in the meshing (relative to meters): e.g.,
 10^{-6} for μm

dimension : dimension of simulation (1,2,3)

temperature : temperature of the system [K]

solve : list of simulations to be executed, in the order of execution; if the list contains “sweep”, a sweep is performed as specified in sweep block in the Solver section.

```
solve = (strain,driftdiffusion,
quantum_electrons, quantum_holes)
```

resultpath : path for output directory

output format : format of the output data: *gmw* for **GMV** , *ise* for **Tecplot** , *grace* for

xmgr (ascii data column type), *vtk* for **Paraview** .

plot : list of output variables which are calculated and available in output files. It may be overridden by defining list specific to one or more modules.

4.5 Output description

At the end of the execution, the program will write the results of the simulation in the directory specified by `resultpath` , with the format specified by `output format`. The output variables are specified in a list plot, in each Module.

TiberCAD output is divided in two classes: **mesh-based** and **mesh-independent** quantities.

Mesh-based quantities are all the quantities associated with the nodes of the mesh, such as Fermi level, electron and hole density, conduction and valence band, etc., together with all the quantities associated with the elements of the mesh, such as current density.

The output values for these quantities are reported in the files `simname msh.ext`, where *simname* is the simulation module used for the calculations and `ext` is the extension of the chosen file format, e.g. `vtu` for paraview output.

```
strain_msh.vtu
```

In the case a sweep calculation is performed and the `plot data` keyword is set to true, the output files are of the kind `simname sweepvariable step value msh.ext`, where

sweepvariable is the variable with respect to which the sweep is performed (e.g. gate voltage) and *step value* is the value of this variable at that step; e.g the result at the step *Vbias = 1.1* will be found in the file:

```
dd_Vbias_1.1_msh.vtu
```

`mesh-independent` quantities are the quantities which are not associated to the mesh, for example current at the contacts of a diode or quantized energy levels in a quantum well. These **mesh-independent** quantities are displayed in separated files, with the format `simname.ext`, e.g `quantum_electrons.dat` , where *simname* is the name of the model (simulation) associated to the results. If a sweep is performed, the output file gets the format `sweep name simname.ext`, where *sweep_name* is the name of the sweep performed, for example

```
sweep_drain_driftdiffusion.dat
```

Inside the file, output values for all the steps of calculation are shown.

4.6 Example of Input file

Here is an example of the input file template:

```
Device hemt1
{
  meshfile = hemt_msh.grd
  mesh_units = 1e-6
  material = GaN
```

```
y-growth-direction = (0,0,0,1)
z-growth-direction = (1,0,-1,0)
x-growth-direction = (-1,2,-1,0)
Region barrier
{
    material = AlGaIn
    x = 0.14
}
Region barrier_doped
{
    mesh_regions = (barrier_dop_s, barrier_dop_d)
    material = AlGaIn
    x = 0.14
    Doping
    {
        density = 1e21
        type = donor
        level = 0.026
    }
}
Region buffer { }

    Region buffer_doped
    {
        mesh_regions = (buffer_dop_s, buffer_dop_d)
        Doping
        {
            density = 1e21
            type = donor
            level = 0.026
        }
    }
Region cap
{
    Doping
    {
        density = 5e18
        type = donor
        level = 0.026
    }
}
Region cap_doped
{
    mesh_regions = (cap_dop_s, cap_dop_d)
    Doping
    {
        density = 1e21
```



```

        type = donor
        level = 0.026
    }
}
Region passivation
{
    mesh_regions = (passiv1, passiv2, passiv3, passiv4)
    material = SiN
}
Cluster semiconductor
{
    regions = -passivation
}
}
Module driftdiffusion
{
    name = dd
    regions = all
    coupling = electrons
    save_state = true
    #load_state = output_Nt7e16_out/dd_Vd_50.tsv
    statistics = FermiDirac
    strain_simulation = macrostrain
    plot = (Ec, Ev, Eg, eQFermi, hQFermi, eDensity, hDensity, Polarization,
           eCurrentDensity, hCurrentDensity, CurrentDensity, ContactCurrents,
           ElPotential, ElField, eMobility, hMobility, eConductivity, hConductivity)

    NonlinearSolver linesearch
    {
        # type = ls
        relative_tolerance = 1e-15
        step_tolerance = 5e-3
        max_iterations = 15
        LinearSolver petsc
        {
            ksp_type = pconly
            preconditioner = lu
        }
    }
}
Physics
{
    recombination (srh, direct, auger) { }
    electron_mobility field_dependent
    {
        region = buffer
        low_field_model = constant
    }
}

```

```
electron_mobility doping_dependent
{
    region = (barrier, barrier_doped, cap, cap_doped, buffer_doped)
}
trap fixed_charge
{
    region = GaN/SiN
    Nt = 2.74e13
}
trap acceptor
{
    region = buffer
    Nt = 7e16
    Et = 0.5
    reference = cb
}
polarization (piezo, pyro) { }
}

Contact drain
{
    voltage = @Vd
}
Contact source
{
    voltage = 0.0
}
Contact gate
{
    regions = (gate1, gate2)
    type = schottky
    work_function = 1.5272
    voltage = @Vg
}
}

Module macrostrain
{
    regions = -passivation # all the device except Region "passivation"
    plot = all
    LinearSolver # default
    {
        tolerance = 1e-7
        max_iterations = 10000
        #xmonitor = true
        pc = composite
    }
}
```

```

Physics
{
    Boundary substrate
    {
        regions = bottom
        material = GaN
        y-growth-direction = (0,0,0,1)
        z-growth-direction = (1,0,-1,0)
        x-growth-direction = (-1,2,-1,0)
    }

    Boundary extended_material
    {
        regions = side
    }
}

Module sweep
{
    name = sweep_g
    variable = Vg
    solve = sweep_d
    start = -1
    stop = 1.0
    steps = 2
    max_step = 0.25
    min_step = 1e-4
    #plot_data = true
}

Module sweep
{
    name = sweep_d
    variable = Vd
    solve = driftdiffusion
    start = 0.0
    stop = 50.0
    steps = 200
    min_step = 1e-4
    plot_data = true
}

Module selfconsistent
{
    name = relaxation
    solve = (macrostrain, driftdiffusion)
}

```

```
absolute_tolerance = 1e-3
relative_tolerance = 1e-5
}
```

Simulation

```
{
  temperature = 300
  verbose = 3
  solve = (macrostrain, dd, sweep_g)
  logfile = output_Nt5e16_Al0.215_SiN/hemt.log
  # these parameters can be defined in modules, too
  resultpath = output_Nt5e16_Al0.215_SiN
  output_format = vtk
}
```

SIMULATION DYE SOLAR CELLS

In this Tutorial we will show how to model and simulate a Dye Sensitized Solar Cell (DSC).

The basic structure of a DSC is the following:

1. A photoanode made of a semiconductor mesoporous medium (TiO_2) plunged into a liquid electrolyte.
2. A region of liquid electrolyte only in contact with a counter-electrode covered by a thin platinum layer.

The surface of the semiconductor is covered by a mono-layer of dye molecules which make the oxide photoactive. The mesoporous material is obtained by sintering together nanoparticles of TiO_2 of 15-20 nm in diameter.

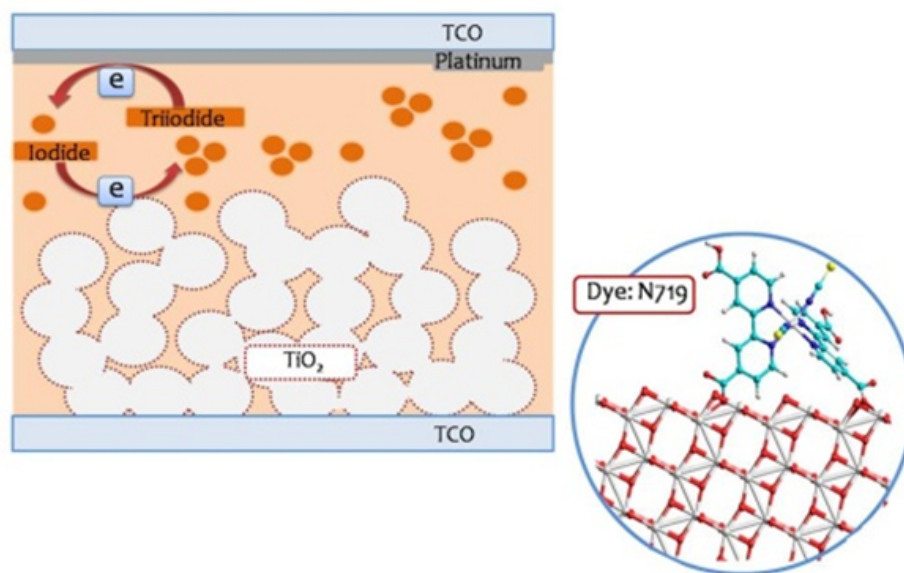
The diameter of the nanoparticles is fundamental to obtain the right porosity of the material, in fact in order to have a high light harvesting from the dye molecules is necessary to have a large effective area that can be obtained only with a porous material, moreover the porosity of the semiconductor allows the electrolyte to be in contact with the dye molecules.

Fig. 1: Scheme of a DSC (in the inset: a typical Dye molecule).

The functioning of the device is the following: when a photon is absorbed by a dye, it excites an electron which is transferred into the conduction band of the porous material. The charge transfer is extremely fast, in the order of tens of fs.

Then, the electron percolates inside the porous material until it reaches the anode contact: a transparent conductive oxide (TCO). The ionized dye is regenerated by the electrolyte via a redox process where iodide is oxidized and triiodide is formed. This regeneration has two important consequences: on the one hand the dye is now ready to absorb another photon and on the other hand the equilibrium of the redox pair concentration in the electrolyte is broken. In particular, the concentration of triiodide is increased at the expense of iodide concentration.

A concentration gradient is formed that moves triiodide ions towards the counter-electrode and iodide ions towards the mesoporous material. Finally, the triiodide ions reach the platinum where they are reduced and transformed back in iodide ions. The cycle of charge carriers is completed and a current has been established inside the cell exploiting the energy of the photon absorbed.



Two important facts must be noted:

1. In the cell there is no permanent chemical transformation;
2. The split of the electron-hole pair created by the photon occurs directly at the dye level, the electron percolates inside the TiO_2 while the hole is handled by the electrolyte. This means that charge carriers in DSCs are majority carriers and the recombination is suppressed. That is the reason why DSCs allow reasonable efficiencies despite they are made of very poor quality materials.

To run this tutorial the following files should be present in the working directory:

`dsc.tib` : input file (physical parameters of the device)

`dsc.msh` : mesh generated by GMSH from the script (geometry of the device)

`dsc.geo` : script for **GMSH**

5.1 Device structure

In this tutorial we will see a simple 1D model, made of two physical regions:

1. TiO_2 region, which is the mesoporous medium where the light absorption and recombination takes place
2. Electrolyte region

Two boundary condition points represent the photoanode (left) and the counter-electrode (right).

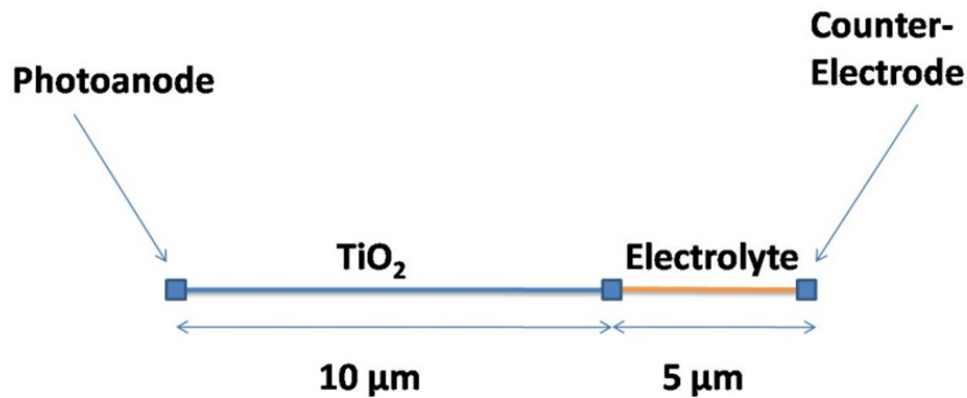


Fig. 2: The scheme of the 1D device.

Definition of dsc.tib

This is the description of the device physical regions:

Device

```
{
  meshfile = bulk.msh

  Region TiO2
  {
    material = TiO2mes
    porosity = 0.5
  }

  Region electrolyte
  {
    material = TiO2mes
    TiO2 = false
  }
}
```

|warn| for both region (:math:`\text{TiO}\_2` and electrolyte) the same material for the TiO2mes), the parameter porosity defines which porosity has the material. The flag TiO2 = false defines instead that in the electrolyte region only the electrolyte is present and there is no semiconductor. By default the dssc module assumes that the device contains both the semiconductor and the electrolyte.

This is the definition of Simulation Models and associated Boundary Conditions:

Module dssc

```
{

  name = dssc
```

```
plot = (Potential, Density, TotalChargeDensity, NetRecombination, Current, EField)

Solver linesearch
{
  max_iterations = 300
  step_tolerance = 1e-4
  linear_solver
  {
    preconditioner = lu
  }
}

Physics
{
  generation = dssc_generation
  beta = 1.0
}

Contact anode
{
  type = ohmic
  bias = $V[0.0]
}

Contact cathode
{
  type = Pt
  load = $R[1e8]
}
}
```

The module dssc contains the name of the simulation (and the list of output plotted), the subsections for the solver, the physics and the contacts. The physics subsection must contain at least one flag for the generation module. The non linear exponent in the electron density recombination is assumed to 1 (linear case), but can be a changed to a different value for non-linear recombination. The contact modules must be two: photoanode and counter-electrode.

The two contacts are type = ohmic (photoanode) and Pt (counter-electrode). The external bias is applied at the photoanode (bias), while the counter-electrode is closed over an external resistance (load). In case it is wanted to simulate a cell under illumination the load must be set to a high value (open-circuit condition) and then switched to a load = 1 value (short-circuit condition).

For a simulation in dark condition, load can be set to load = 1 immediately.

The contact modules can include the flag region = (region_1, region_#) in case that contact (cathode or anode) is the composition of different mesh regions.


```

Module dssc_generation
{
    regions = TiO2
    plot = (Distance, Generation)
    light_direction = (1, 0, 0)
    light_intensity = $x
    dye = N719

    Contact anode
    {
    }
}

```

The Module **dssc_generation** defines the photogeneration within the device. The generation model must define which is the region(s) where photogeneration occurs (regions), the intensity of light (light_intensity), the direction of the light rays (light_direction), the kind of Dye used in the cell (dye) and the boundary region where light has the maximum intensity (in our example “anode”).

Another flag illumination_spectrum = <material> can be set if we want to change the solar spectrum for generation (for example in case the light source we want to simulate is not the conventional sun spectrum).

By default the flag is set to illumination_spectrum = Sun1p5am conventional sun spectrum 1.5 AM.

```

Module sweep
{
    name = sweep_gen
    solve = (dssc_generation, dssc)
    variable = $x
    values = (0, 1e-12, 1e-11, 1e-10, 1e-9, 1e-8, 1e-7, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2)
    plot_data = true
}

```

```

Module sweep
{
    name = sweep_R
    solve = dssc
    variable = $R
    values = ( 1000, 100, 10, 1 )
    plot_data = true
}

```

```

Module sweep
{
    name = sweep_V
    solve = dssc
    variable = $V
}

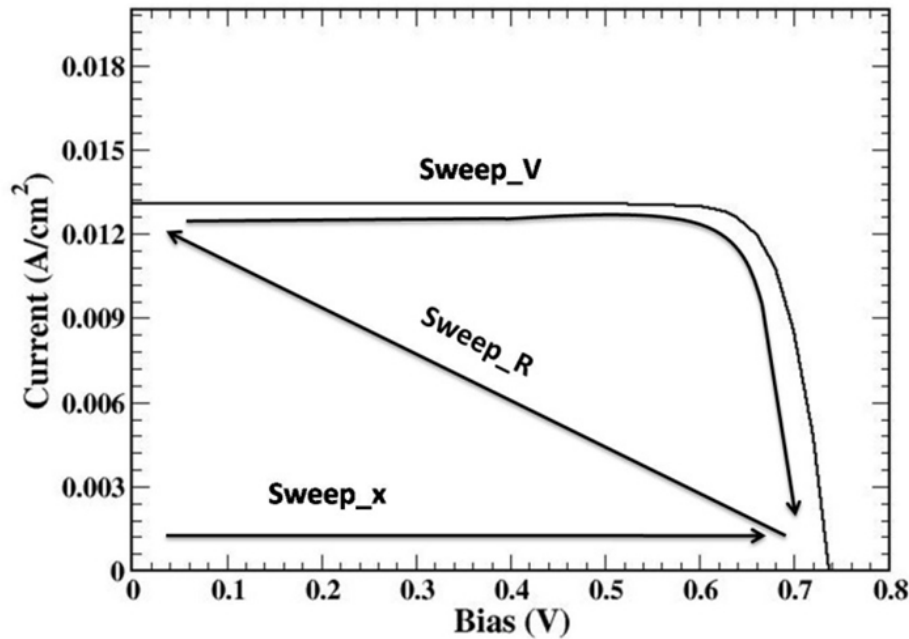
```

```

values = (0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.62, 0.64, 0.66, 0.68, 0.7, 0.72, 0.74, 0.76, 0.78, 0.8)
plot_data = true
}

```

The three sweeps needed to solve the cell: the first sweep is needed to push the device to open circuit condition under illumination, the second sweep switches the state of the cell from open circuit condition (high external load) to short circuit condition (load = #), then the final sweep for the bias. In figure the three sweeps of the code are shown:



The definition of model-independent parameters of the Simulation:

```

Simulation
{
  searchpath = ../../materials
  solve = (sweep_gen, sweep_R, sweep_V)
  resultpath = output
  output_format = grace
}

```

The **output** of the calculation is set in the module dssc section.

We can chose to plot the internal potential profiles (flag Potential) which includes electrostatic potential, all the densities (flag Density) which includes electron, iodide, triiodide, cation densities and recombination and generation profiles, all the currents (flag Current) which include the x, y and z components of the currents (electronic, iodide, triiodide and cation, total current) and the electric field.

Finally the flag (ContactCurrents) allows the plot of the current flowing through the contacts.



In a DSC the current I_s is fundamentally diffusion driven, the electric field and the consequent drift current is rather small.

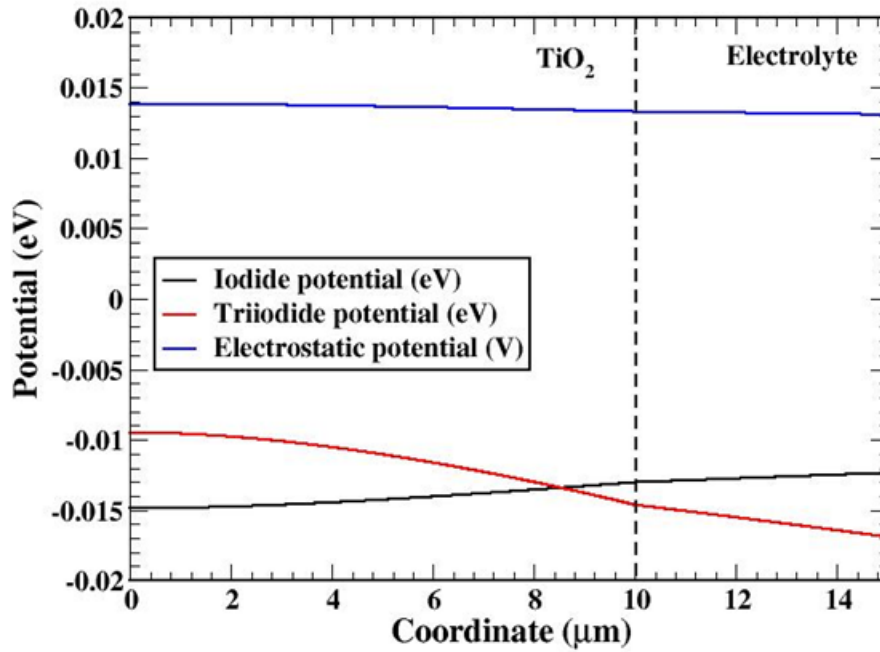
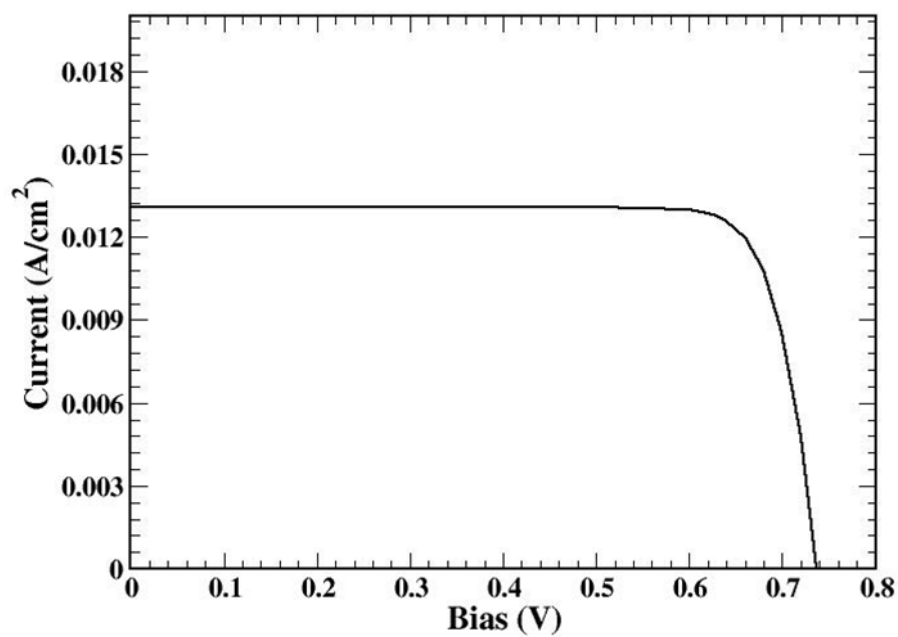
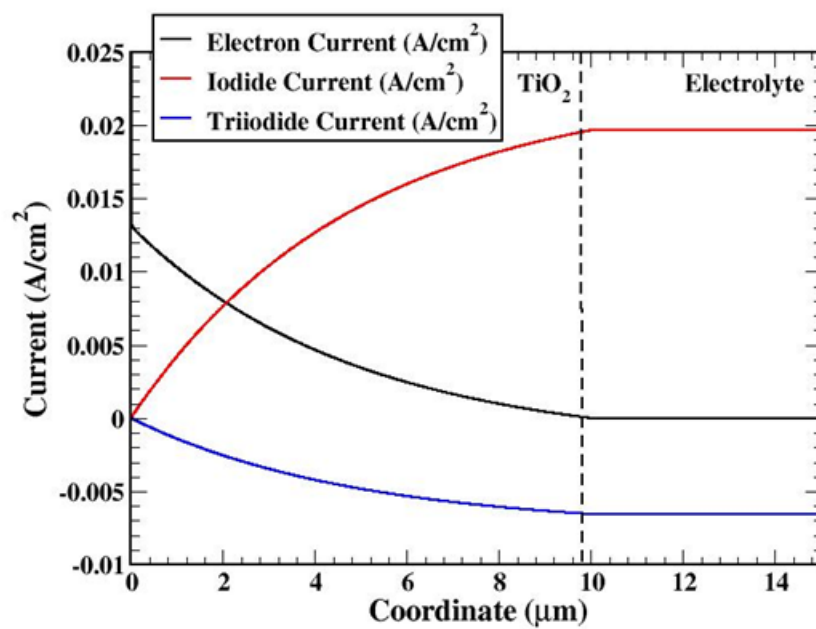


Fig. 4: Potential profiles for the ionic species and the electrostatic potential (short-circuit condition).

Fig. 5: Current contributions within the cell (short-circuit condition).

Fig. 6: I-V characteristic of the device.



Part II

Glossary

modules This is an example of the Syntax view

See *modules* for an overview over Modules option.

constant charge a constant charge can be assigned by specifying only the sheet carrier density N_s in cm^{-2} . The sheet charge density will then equal N_s multiplied by the elementary charge e . A positive N_s produces a positive surface charge.

electronic surface states in this case the surface charge is produced by electrons occupying a surface state with a density of states in form of a delta function. The density of occupied states then reads

degeneracy degeneracy of the quantum state

BIBLIOGRAPHY

- [Kalyanasundaram] Kalyanasundaram Kuppaswamy, “Dye-Sensitized Solar Cells”, *Editor Crc Pr Inc.* , 2010.
- [Gagliardi] Alessio Gagliardi, Matthias Auf der Maur, Desiree Gentilini and Aldo Di Carlo, “Modeling of Dye sensitized solar cells using a finite element method”, *J. Computational Electronics* , vol. 8, pp. 12, 2009.
- [Wachutka] Wachutka, IEEE Trans CAD Cicui. Integr and Syst. (1990)].
- [Povolotskiy] Povolotskiy et al. JAP (2006).
- [Wang] Wang et al. Science (2006).
- [Gao] Gao et al NanoLetter (2009).
- [Selberherr] Siegfried Selberherr, Analysis and Simulation of Semiconductor Devices, SpringerVerlag Wien New York, 1st edition, 1984.
- [Wachutka] Gerhard K. Wachutka, Rigorous thermodynamic treatment of heat generation and conduction in semiconductor device modeling,” IEEE Transaction on Computer-aided Design., vol. 9, pp. 11, 1990.

INDEX

B

Bulk Si

- input, 9
- meshing, 9
- modeling, 7
- output, 12
- run, 12

C

- characteristic
 - n-MosFET 2D, 25
- constant charge, 67

D

- degeneracy, 67
- device
 - n-MosFET 2D, 22
 - Si-Pn Diode, 15

E

- electronic surface states, 67

I

- input
 - Bulk Si, 9

M

- meshing
 - Bulk Si, 9
- model
 - Si-Pn Diode, 15
- modeling
 - Bulk Si, 7
- modules, 67

N

- n-MosFET 2D
 - characteristic, 25
 - device, 22
 - output, 27
 - physic, 26
 - simulation, 23

O

- output
 - Bulk Si, 12
 - n-MosFET 2D, 27
 - Si-Pn Diode, 20

P

- physic
 - n-MosFET 2D, 26
 - Si-Pn Diode, 19

R

- run
 - Bulk Si, 12
 - Si-Pn Diode, 19

S

- Si-Pn Diode
 - device, 15
 - model, 15
 - output, 20
 - physic, 19
 - run, 19
 - solver, 18
- simulation
 - n-MosFET 2D, 23
- solver

Si-Pn Diode, [18](#)